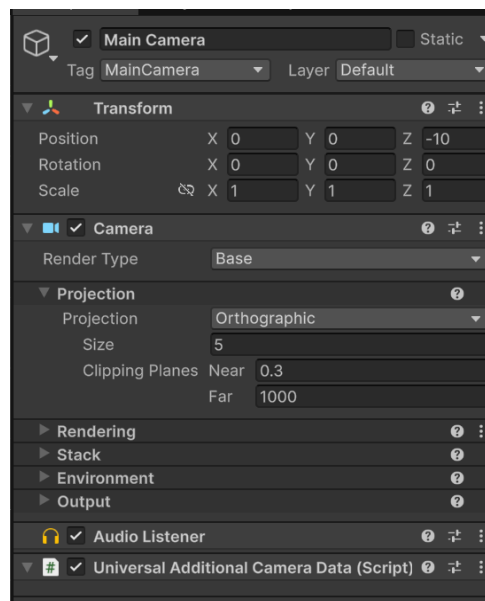


Technical Document_Lab.1

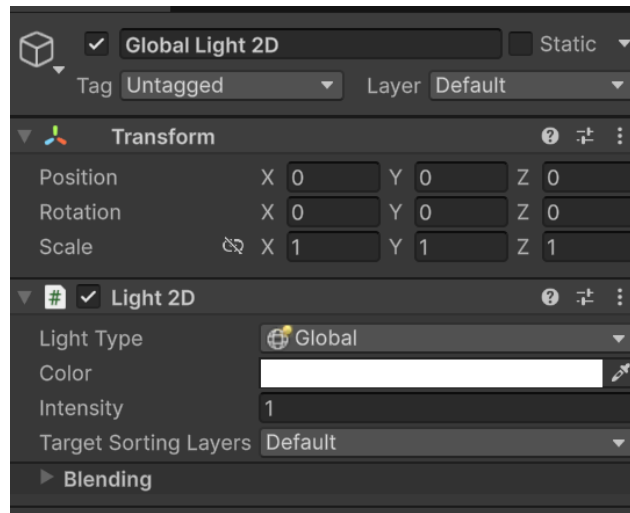
In this document we will be analyzing a pre-made unity project, as well as going over our experience making small modifications to it and building a scene from existing assets.

“Level 1” – Quick scene, game object and component analysis



The Camera object allows the player to see what's going on inside of the game.

Generic Game Object component – Transform: used to alter the position, rotation and scale of the object within the provided 3D space. Some game objects do not have or lock the transform component.



The Global Light 2D object allows for the simulation of lights and shadows of every object in the scene equally. With the use of Normal Maps, shadows may be rendered over specific Sprites/Objects.

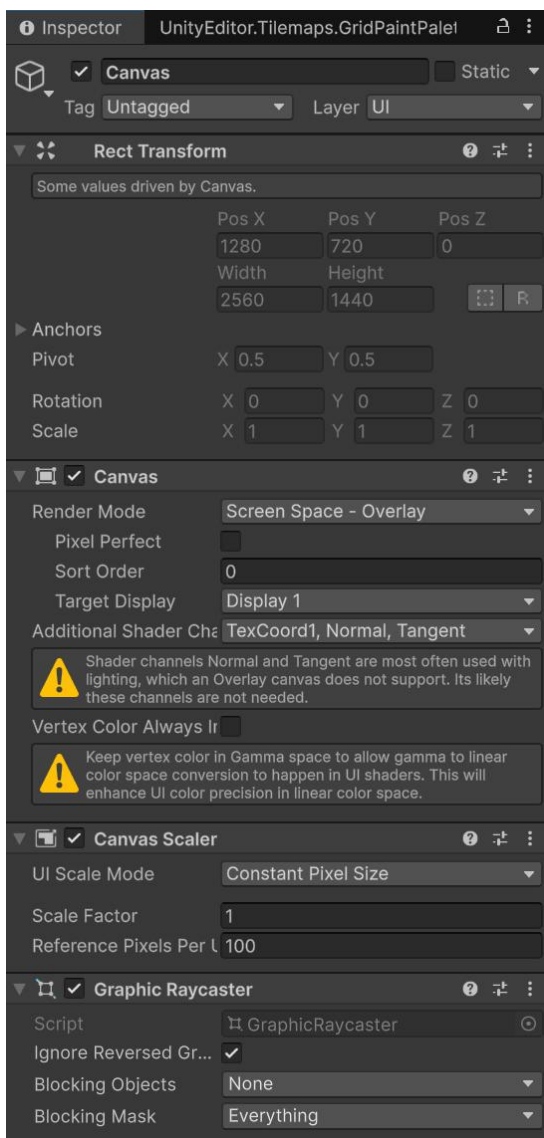


Fig. A

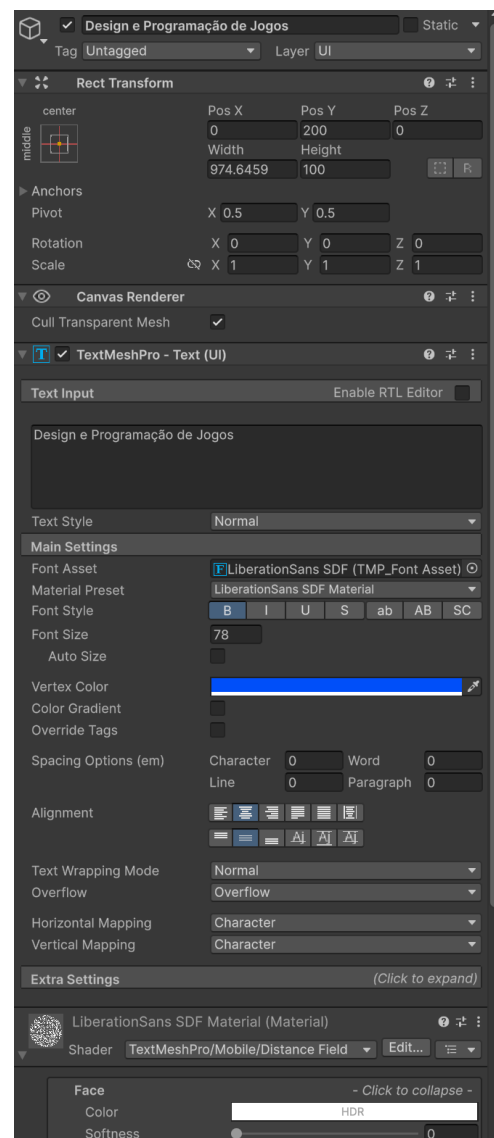


Fig. B

Fig. A – The canvas object serves as a general parent, renderer and handler of User Interface elements.

The Canvas Scaler component changes the resolution of the Canvas's zone.

Fig. B – In a Text object, parented to the aforementioned Canvas object, any type of text can be written, the Rect Transform acts the same as the generic Transform component, with the addition of the Pivot property which determines where the interface will be centered, in this object there's also a shader that alters the way the text is presented to the viewer/player.

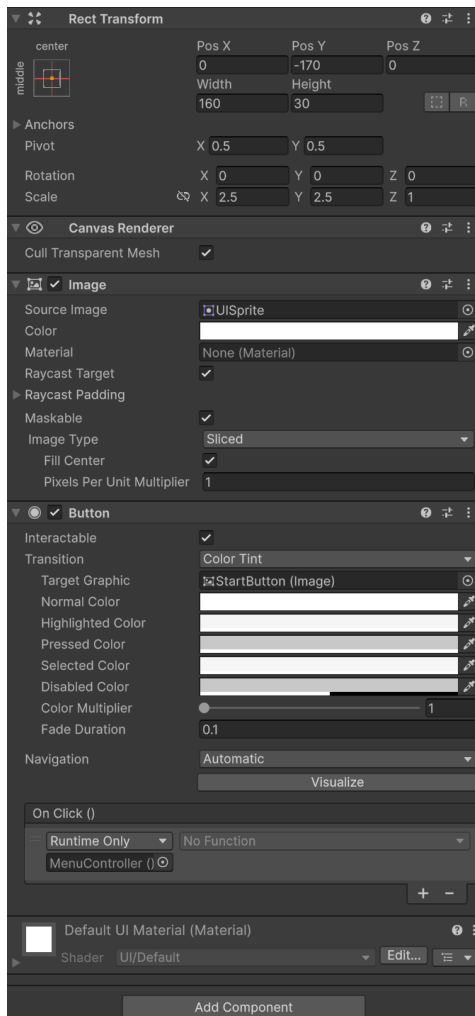


Fig. C

Fig. C – The Button Object shares some components with the Text Object, with the addition of having one, or more, event(s) associated to it. In this case, one which detects if the button is clicked by the player.

Fig. D – The Event System object is an empty Game Object with an Event System component associated to it. This component serves as a listener for Event triggers, allowing every element that contains events to communicate. It also determines what inputs are active and what objects are selected.

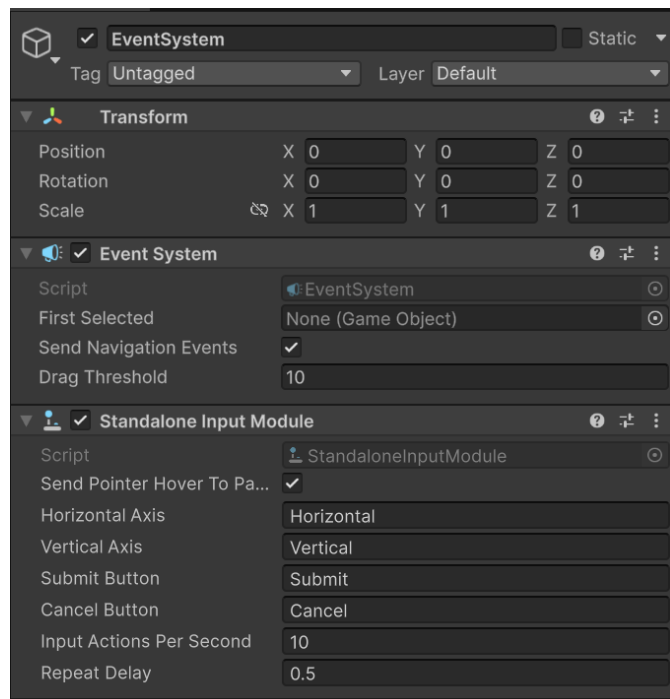


Fig. D

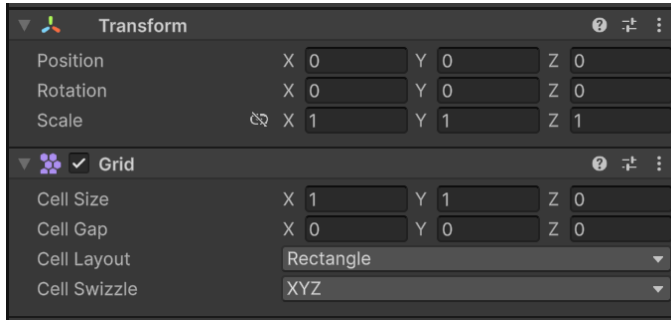


Fig. E

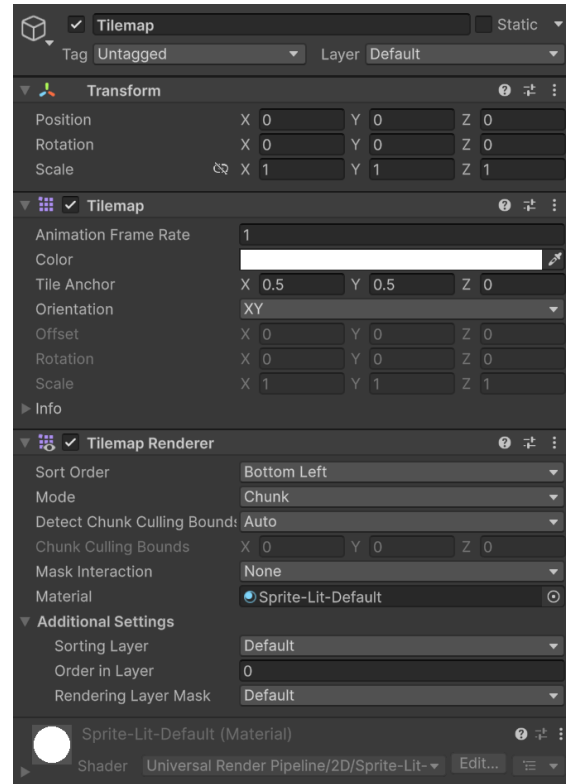
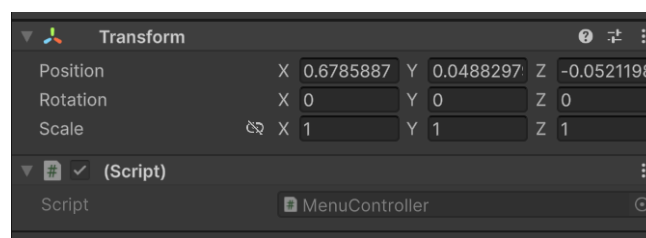


Fig. F

Fig. E - The Grid Component defines a zone which allows for placement of Visual Assets, primarily using a TileMap component, aiding in its alignment.

Fig. F – A TileMap uses the parameters within a Grid component to enable the painting of tiles directly onto the scene. The Tilemap Renderer serves to customize the object's appearance, as well as how the tiles are rendered/grouped.



The Menu Controller object is, similarly to the Event System, an empty Game Object with the addition of a Script (see below for content) component associated which allows the player to interact with the interface.

```

# Menu Controller (Mono Script)

Assembly Information
Filename Assembly-CSharp.dll

using UnityEngine;
using UnityEngine.SceneManagement;

public class MenuController : MonoBehaviour
{
    public string levelSceneName = "Level1";

    public void StartGame()
    {
        // Reiniciar score ao iniciar
        if (GameManager.Instance != null)
            GameManager.Instance.ResetScore();

        SceneManager.LoadScene(levelSceneName);
    }

    public void QuitGame()
    {
        #if UNITY_EDITOR
            UnityEditor.EditorApplication.isPlaying = false;
        #else
            Application.Quit();
        #endif
    }
}

```

Script analysis – This script allows for two different interactions: one which loads a different scene, based on the *levelSceneName* string (text) variable and another which closes the game, or, if in the unity game engine, stops the game simulation.

“Level 2” – Level creation

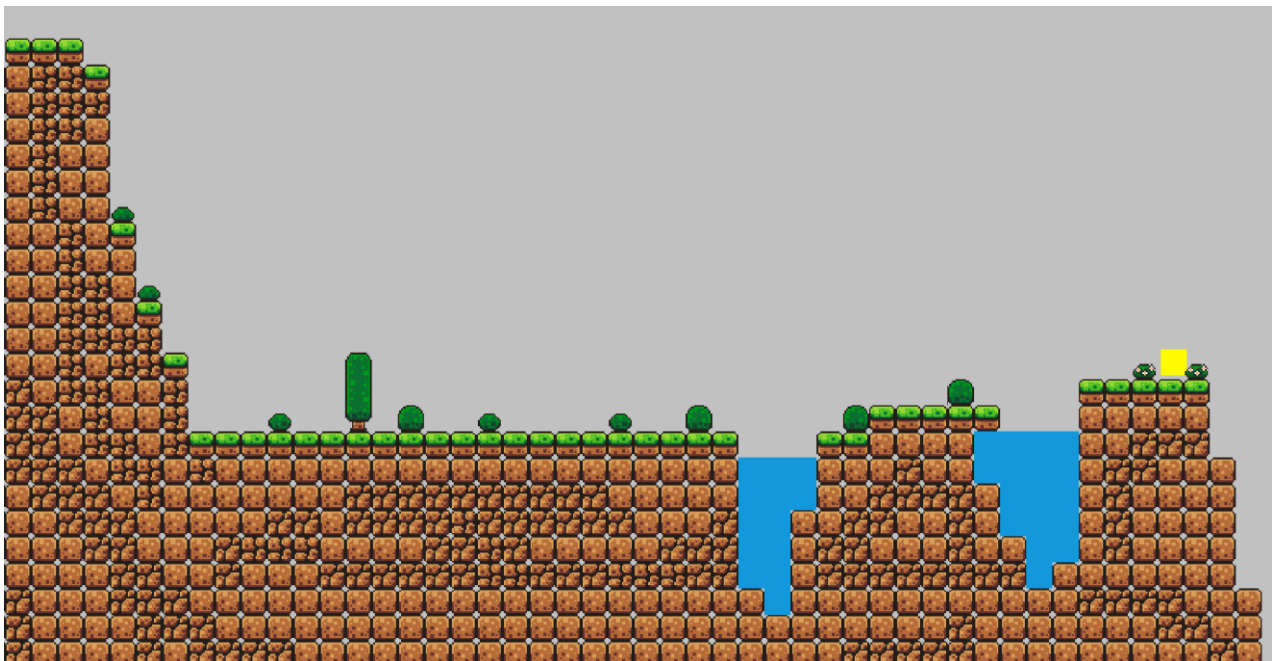


Fig. G – Design of a short level

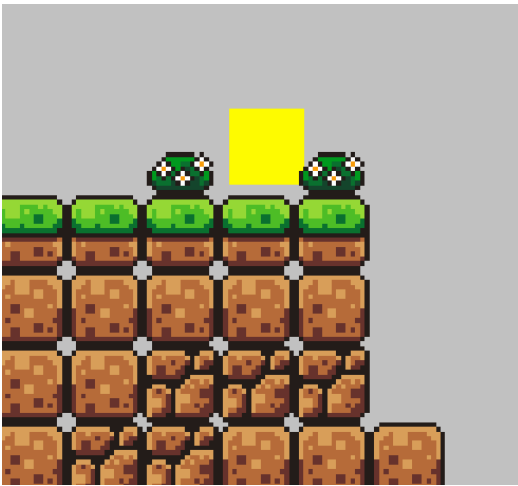


Fig. H

```

# Collectible (Mono Script)

Assembly Information
Filename Assembly-CSharp.dll

using UnityEngine;

public class Collectible : MonoBehaviour
{
    public int scoreValue = 1;
    public bool destroyOnCollect = true;

    private void OnTriggerEnter2D(Collider2D collision)
    {
        if (collision.CompareTag("Player"))
        {
            if (GameManager.Instance != null)
            {
                GameManager.Instance.AddScore(scoreValue);
            }
            else
            {
                Debug.LogWarning("GameManager não encontrado na cena. Certificar que existe um GameManager com script anexado.");
            }

            if (destroyOnCollect)
                Destroy(gameObject);
            else
                gameObject.SetActive(false);
        }
    }
}

```

Fig. I

Script Analysis (Figure I) – This script waits for the `OnTriggerEnter2D` event, which triggers on collision, and then checks if the object which entered the `Collision2D` area has the tag “Player”, on which case it will check if a game manager instance is running and, if it is, fire the `AddScore` event with the value of the integral (Number) variable “scoreValue”. Lastly, depending on whether the Boolean variable “destroyOnCollect” is true or false, it will either destroy the collectible object or hide it, respectively.

“Level 3” – Rigidbody component, different Body Types

Body Types:

- **Dynamic**- The Dynamic body is the most common `Rigidbody2D` body type for things that need movement since it interacts with all other body types and is, by default, affected by gravity and forces. This body type is the heaviest of performance.
- **Kinematic**- The Kinematic body is not normally affected by gravity or other forces hence it requires additional programming. This approach is lighter on performance and may allow for more customizability.
- **Static**- The Static body behaves as an immovable object. This is the lightest body type when it comes to performance.

“Level 4” – Quick resume of steps taken

Firstly, we created a text label using TextMeshPro, adding it into the canvas to be used as a Score display. We then created an empty game object named Game Manager, in which we inserted the GameManager script as a component. This script handles, through the use of events, the update of the integral variable “Score” as well as the previously created text label which we set as the target for the object Variable “scoreText”.

Secondly, we created a respawn system with empty game objects and colliders. Starting with the Respawn Trigger object, with the SimpleRespawn scripts as one of its components, which detects collisions with the BoxCollider2D component and sets the player’s transform parameters to the ones of its Transform Variable “respawnPoint”, to which we link a new game object.

Finally, we went back to the Menu scene and tested the full gameplay loop, making small adjustments as needed.

Sources:

- The official unity documentation for all needed use cases, namely:

<https://docs.unity3d.com/6000.0/Documentation/Manual/tilemaps/tilemaps.html>

<https://docs.unity3d.com/ScriptReference/Grid.html>

<https://docs.unity3d.com/ScriptReference/UIElements.Button.html>

<https://docs.unity3d.com/ScriptReference/RigidbodyType2D.html>

- The official unity manual for the global light 2d game object:

- <https://docs.unity3d.com/Packages/com.unity.render-pipelines.universal@7.1/manual/2DLightProperties.html>